

TEAM – 03

MEDIQUEUE

INTELLIGENT APPOINTMENT PRIORITIZATION SYSTEM

Name:PUNYA S Y

Sec : MCA 'A'

USN:24SUPMCAGL062

TABLE OF CONTENT

CHAPTER NO	DESCRIPTION	PAGE NO
1	Introduction	1
2	Problem Statement	2
3	Objectives	3
4	System Analysis 4.1 Existing System 4.2 Limitations of Existing System 4.3 Proposed System	4
5	System Design 5.1 Architecture	6
6	Database Design	7
7	Module Description	8
8	Implementation Details	11
9	System Workflow	13
10	Testing and Results	14
11	Future Enhancement and Conclusion	15

CHAPTER 1

INTRODUCTION

In modern healthcare systems, managing appointments efficiently has become a critical challenge. Hospitals and clinics handle hundreds of patients daily, and treating them in the order of arrival is not always ideal. Emergency or high-priority patients often experience delays, while minor cases may occupy valuable consultation slots.

MediQueue – Intelligent Appointment Prioritization System aims to overcome these inefficiencies by introducing an AI-powered scheduling mechanism that automatically prioritizes appointments based on medical urgency, patient demographics, and doctor availability.

This system goes beyond traditional booking platforms by integrating a **priority scoring engine** that intelligently ranks patients according to urgency level, age, medical condition, and symptoms. It helps doctors focus on critical cases first while maintaining fairness and transparency in patient scheduling.

MediQueue integrates seamlessly into existing hospital management systems and provides a user-friendly web interface accessible to **patients**, **doctors**, and **administrators**. The system leverages **Spring Boot (Java)** for the backend, **MySQL** for data persistence, and **HTML/CSS/Bootstrap** with **Thymeleaf** for a responsive front-end interface.

Key Features:

- Automated appointment prioritization using AI logic.
- Doctor dashboards showing high-to-low priority patients.
- Real-time updates and dynamic rescheduling.
- Integration with existing appointment systems.
- Transparent communication with patients about their queue position.

The system is designed to enhance operational efficiency, reduce patient waiting time, and support data-driven decision-making in healthcare scheduling.

CHAPTER 2

PROBLEM STATEMENT

Traditional hospital appointment systems operate on a **first-come, first-served** model, treating all patients equally regardless of urgency. This causes several operational issues such as:

- **Delay for emergency cases:** Critical patients wait long periods despite urgent needs.
- **Manual workload:** Doctors and staff manually organize schedules.
- **Lack of fairness:** Urgent or elderly patients have no prioritization advantage.
- **No predictive logic:** The system cannot estimate waiting time or criticality.
- **Patient dissatisfaction:** Long queues and delays reduce trust and experience.

Hence, there is a pressing need for a **smart, rule-based prioritization system** that dynamically orders appointments using intelligent logic, ensuring urgent and critical patients receive timely medical attention.

CHAPTER 3

OBJECTIVES

The major objectives of MediQueue are:

1. To automate hospital appointment scheduling using AI-driven logic.
2. To prioritize patients based on severity, urgency, and age.
3. To develop doctor and admin dashboards for real-time monitoring of patient queues.
4. To enhance communication by providing real-time updates on appointment order and waiting time.
5. To reduce human intervention and manual scheduling errors.
6. To improve hospital efficiency and patient satisfaction through optimized workflows.
7. To provide integration support with existing healthcare management systems.
8. To ensure secure, scalable, and user-friendly system design.

CHAPTER 4

SYSTEM ANALYSIS

4.1 Existing System

In the current healthcare appointment models:

- Appointments are handled on a first-come basis.
- No distinction is made between emergency and routine cases.
- Scheduling is managed manually or semi-automatically.
- No algorithmic or AI-based priority scoring.
- Limited feedback or communication with patients regarding status.

4.2 Limitations of the Existing System

- High waiting times for critical cases.
- Increased workload for hospital staff.
- Poor utilization of doctor time and resources.
- Absence of a centralized intelligent prioritization mechanism.
- Limited scalability and analytics capability.

4.3 Proposed System

The **MediQueue** system resolves these limitations by:

- Implementing a **rule-based AI prioritization engine**.
- Assigning each patient a **priority score** based on urgency level, symptoms, and other conditions.
- Sorting appointments dynamically based on these scores.
- Allowing doctors and admins to visualize the appointment queue efficiently.
- Providing patients real-time feedback on their estimated waiting time and position.

This ensures an optimized, fair, and efficient healthcare workflow.

CHAPTER 5

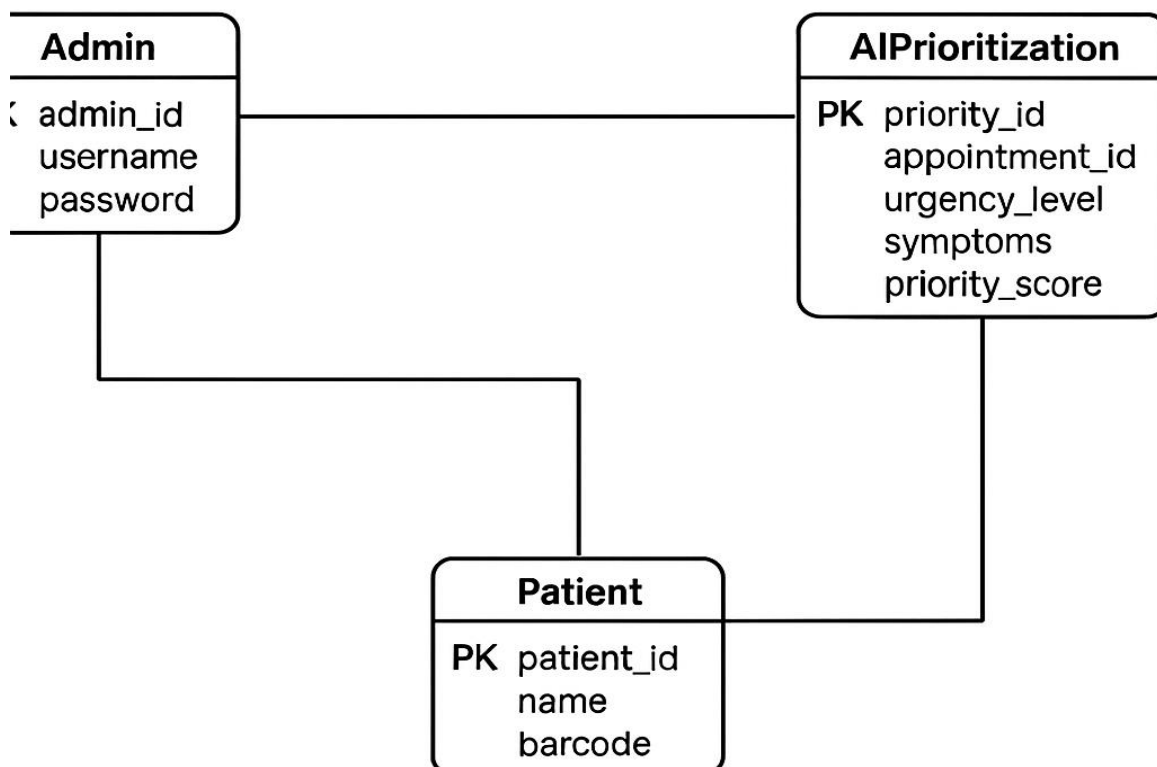
SYSTEM DESIGN

5.1 Architecture

MediQueue follows the **Model-View-Controller (MVC)** architecture:

- **Model:** Entities such as Patient, Doctor, Appointment, and Priority define the data layer using JPA (Hibernate).
- **View:** Built using Thymeleaf, Bootstrap, and JavaScript for interactive dashboards and patient portals.
- **Controller:** REST-based controllers in Spring Boot handle user requests and route them to services.
- **Service Layer:** Contains business logic and AI prioritization algorithms.
- **Database:** MySQL is used to persist structured data like appointments, users, and priority scores.

MEDIQUEUE



5.2 AI Prioritization Logic

The AI logic assigns a **priority score** to each appointment using simple rules:

Condition	Score
Emergency case	+50
Senior citizen (Age > 60)	+20
Chronic disease	+15
New patient	+10
Early booking or long wait	+5

Final Priority = Sum of all applicable scores. Appointments are sorted in descending order of **priority_score**.

CHAPTER 6

DATABASE DESIGN

Database Name: mediqueue_db

6.1 Entity Relationship Overview

- **Patient** books an **Appointment** through the system.
- Each **Appointment** has a corresponding **AI Prioritization** record used to calculate urgency and severity.
- **Doctor** reviews appointments ordered by priority score for efficient handling.
- **Admin** manages all users, doctors, and appointment rules to ensure smooth system operation.

Relationships:

- One **Patient** → Many **Appointments**
- One **Doctor** → Many **Appointments**
- One **Appointment** → One **AI Prioritization Record**

6.2 Tables and Attributes

patients

Column	Type	Description
patient_id	INT (PK)	Unique patient ID
name	VARCHAR(100)	Patient's full name
email	VARCHAR(100)	Contact email (used for login)
password	VARCHAR(255)	Encrypted password for authentication
age	INT	Age of the patient
gender	VARCHAR(10)	Gender of the patient
contact_number	VARCHAR(20)	Patient's contact phone number
symptoms	VARCHAR(255)	Short description of symptoms

doctors

Column	Type	Description
doctor_id	INT (PK)	Unique doctor ID
name	VARCHAR(100)	Doctor's name
specialization	VARCHAR(100)	Field of specialization (e.g., Cardiology, ENT)
email	VARCHAR(100)	Doctor's email address
password	VARCHAR(255)	Hashed password for secure login
availability_status	VARCHAR(50)	Availability of the doctor (Available/On Leave)

appointments

Column	Type	Description
appointment_id	INT (PK)	Unique ID for each appointment booking
patient_id	INT (FK)	Linked to patients(patient_id)
doctor_id	INT (FK)	Linked to doctors(doctor_id)
appointment_date	DATE	Date of appointment
appointment_time	TIME	Scheduled time of appointment
status	VARCHAR(50)	Current status (Booked / Completed / Cancelled)
created_at	TIMESTAMP	Automatically records creation date and time

Column	Type	Description
priority_id	INT (PK)	Unique priority record ID
appointment_id	INT (FK)	Linked to appointments(appointment_id)
urgency_level	ENUM('Low','Medium','High')	Self-reported urgency by patient
symptoms	VARCHAR(255)	Reported symptoms for AI analysis
chronic_condition	BOOLEAN	Indicates if the patient has a chronic illness
age_factor	INT	Weighted score based on patient's age
symptom_severity	INT	AI-generated severity score from symptom keywords
priority_score	INT	Final AI-calculated score used to order appointments

ai_prioritization

admin_users

Column	Type	Description
admin_id	INT (PK)	Unique admin ID
username	VARCHAR(100)	Admin username
password	VARCHAR(255)	Secure hashed password
role	VARCHAR(50)	Role type (Super Admin / Staff)

SELECT * FROM DOCTORS;

IS_AVAILABLE	DOCTOR_ID	EMAIL	NAME	PHONE	SPECIALIZATION
TRUE	1	DanielDavis36@gmail.com	danieldavis	735736	Cardiologist
TRUE	2	DanielDavis36@gmail.com	Dr. Alex Anderson	7566	Psychiatrist
TRUE	3	punyagowda36@gmail.com	punya gowda	08217829492	Orthopedist

(3 rows, 1 ms)

SELECT * FROM PATIENTS;

AGE	DOB	HAS_CHRONIC_DISEASE	LAST_VISIT	REGISTRATION_DATE	PATIENT_ID	ADDRESS	CHRONIC_CONDITIONS	EMAIL
28	2025-11-14	FALSE	null	2025-11-13	1	B.seehalli, 571101, Mysore, Karnataka, India	null	ram123@
40	2025-11-14	TRUE	null	2025-11-13	2	Mysore, Karnataka, India	null	prathika@
50	2025-11-15	TRUE	null	2025-11-13	3	null	null	yashu123@
60	2025-11-14	TRUE	null	2025-11-13	4	B.seehalli, 571101, Mysore, Karnataka, India	null	ram123@

(4 rows, 3 ms)

Edit

CHAPTER 7

MODULE DESCRIPTION

7.1 User Module

7.1 Patient Module

- Registration and login.
- Appointment booking with urgency selection.
- View appointment priority score and estimated waiting time.
- Receive notification upon confirmation or delay.

7.2 Doctor Module

- Login dashboard displaying appointments by priority.
- Mark appointments as completed or reschedule them.
- Filter patients by urgency or time slot.
- View patient history and reports.

7.3 Admin Module

- Manage users (patients, doctors).
- Approve doctor registrations.
- Configure and update priority scoring rules.
- View system analytics, patient load, and appointment trends.

CHAPTER 8

IMPLEMENTATION DETAILS

8.1 Backend

- Framework: **Spring Boot (Java)**
- ORM: **Spring Data JPA (Hibernate)**
- Database: **MySQL**
- Logic Layer: **Priority Scoring Algorithm**
- API Layer: RESTful endpoints (/api/appointments, /api/priorities)

8.2 Frontend

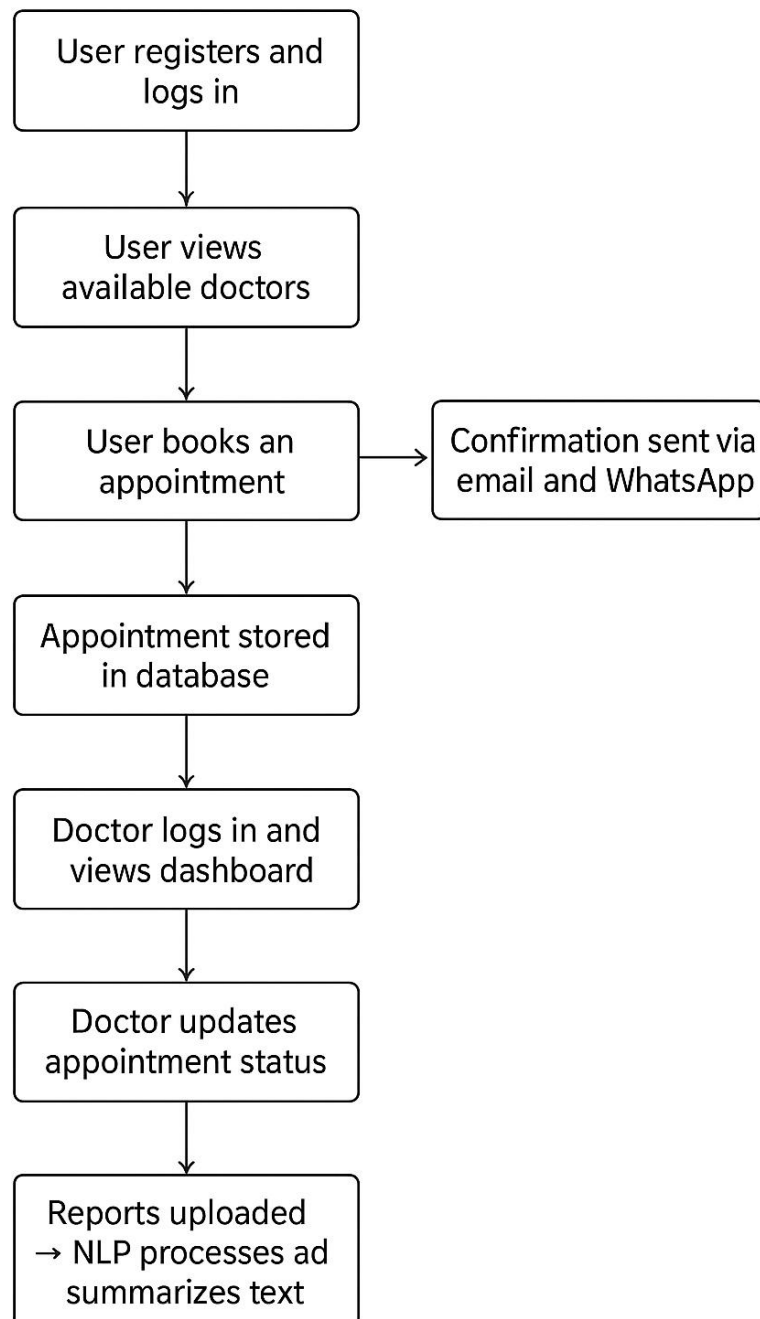
- Tools: **HTML5, CSS3, Bootstrap, Thymeleaf**
- Interactive dashboards and booking forms.
- Mobile-friendly responsive design.

8.3 Example REST Endpoints

Method	Endpoint	Description
GET	/api/appointments	Fetch all appointments
GET	/api/appointments/prioritized	Get sorted appointments
POST	/api/appointments	Create new appointment
PUT	/api/appointments/{id}	Update status or details

CHAPTER 9

SYSTEM WORKFLOW



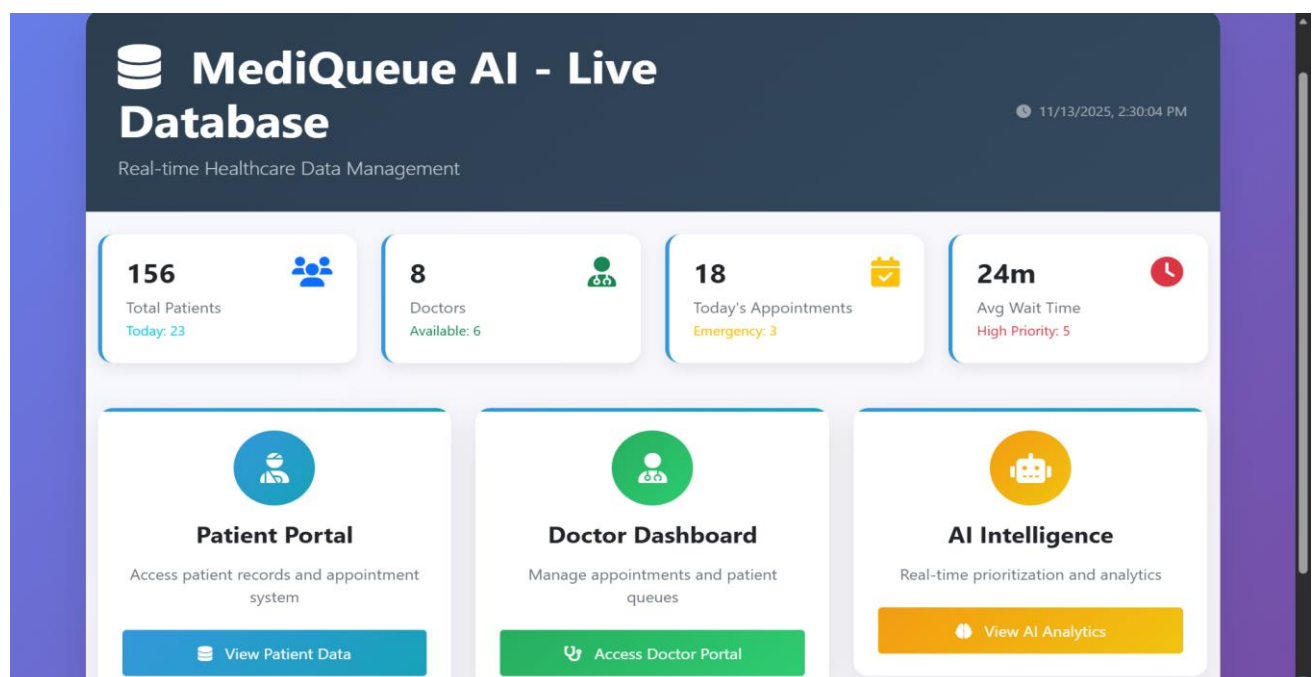
CHAPTER 10

TESTING AND RESULTS

MediQueue - Intelligent Appointment Prioritization System

Module	Test Performed	Expected Output	Result
Admin	View Prioritized List	High → Medium → Low	Success
Doctor	Next Patients Priority	Sorted by Priority	Success
Patient	Select Urgency	Urgency Set	Success
Priority Algorithm	Calculate Priority	Priority Score	Success
Appointment Scheduling	Schedule Appointment	Optimized Slot	Success

OUTPUT :



← Back to Dashboard

Doctor Management

Manage medical team and appointments

Refresh

+ Add Doctor

Medical Team3 Doctors

Search doctors...

danieldavis

Available

👤 Cardiologist

✉ DanielDavis36@gmail.com

☎ 735736

ID: 1

✎

📅

🗑

Dr. Alex Anderson

Available

👤 Psychiatrist

✉ DanielDavis36@gmail.com

☎ 7566

ID: 2

✎

📅

🗑

punya gowda

Available

👤 Orthopedist

✉ punyagowda36@gmail.com

☎ 08217829492

ID: 3

✎

📅

🗑

← Back to Dashboard

AI Analytics

Real-time prioritization and predictive analytics

AI ACTIVE

2:31:37 PM

💡 AI Insights

8 critical cases need immediate attention. AI analyzed 15 appointments.

Refresh AI

Export

AI-Optimized Priority Queue

Patient	Symptoms	Doctor	Priority	AI Score	Predicted Wait	Actions
Emily Brown	Chest pain and shortness of breath	Dr. Daniel Davis	CRITICAL	100	5-10 min	✎
Michael Jones	Chest pain and shortness of breath	Dr. Henry Harris	CRITICAL	100	5-10 min	✎
John Smith	Chest pain and shortness of breath	Dr. Henry Harris	CRITICAL	100	5-10 min	✎

AI Priority Rules

EMERGENCY

Emergency symptoms

+50

AGE

Age > 60 years

+20

CHRONIC_DISEASE

Has chronic disease

+15

SYMPTOM_SEVERITY

High severity symptoms

+25

PEDIATRIC

14 | Page

CHAPTER 11

FUTURE ENHANCEMENTS AND CONCLUSION

Future Enhancements

- Integration of Machine Learning for predictive triage scoring.
- Mobile app for patient-side appointment tracking.
- Voice-based AI assistant for symptom entry.
- Integration with wearable IoT devices for live vitals.
- Cloud-based scalability using AWS/Azure.
- Smart analytics dashboard for doctors and hospital management.

Conclusion

MediQueue represents the next step in intelligent healthcare management. By integrating AI-driven prioritization, it addresses the inefficiencies of manual appointment systems and ensures that critical patients receive the care they need first.

The system's modular architecture, rule-based AI logic, and integration-ready design make it a scalable, future-proof solution for modern healthcare environments. It enhances doctor efficiency, reduces patient waiting times, and promotes fairness and transparency — advancing the vision of **smart, data-driven healthcare**.